

Lab Experiment – 4

Anshul(241210020)

Aim:

To study the compilation and debugging process of a C/C++ program using the GNU Debugger (GDB), and to understand program execution through breakpoints, step-by-step tracing, and runtime analysis.

Theory:

1. Introduction to GNU Debugger (GDB)

GNU Debugger (GDB) is a powerful debugging tool used to monitor, control, and analyze the execution of programs. It helps programmers identify logical errors, execution flow, and variable changes during runtime.

Programs are compiled using the **-g** option to include debugging symbols:

g++ -g program.cpp -o output

2. Program Used in the Experiment

In this experiment, a multiplication table program was compiled and executed using GDB. The program accepts a number from the user and displays its multiplication table.

- User enters a number
- Program processes input
- Loop executes repeatedly
- Multiplication results are displayed

3. GDB Commands Used

Breakpoint:

`break main`

This command pauses execution at the main function.

Run:

`run`

Starts the program inside GDB.

Step:

step

Executes one statement at a time.

Continue:

continue

Resumes execution until the next breakpoint or program termination.

List:

list

Displays source code around the current execution point.

Code used :-

Using “list” command

```
30
[Inferior 1 (process 6756) exited normally]
(gdb) list
1      #include<bits/stdc++.h>
2      using namespace std;
3
4      int main() {
5          int n;
6          cout<<"enter number : ";
7          cin>>n;
8          cout<<"\n";
9          cout<<"Table of "<<n<<" : \n"<<endl;
10         for(int i=0;i<10;i++){
(gdb) print n
No symbol "n" in current context.
(gdb) run
Starting program: /home/nitdelhi/coa
Breakpoint 1, main () at anshul_coa.cpp:4
```

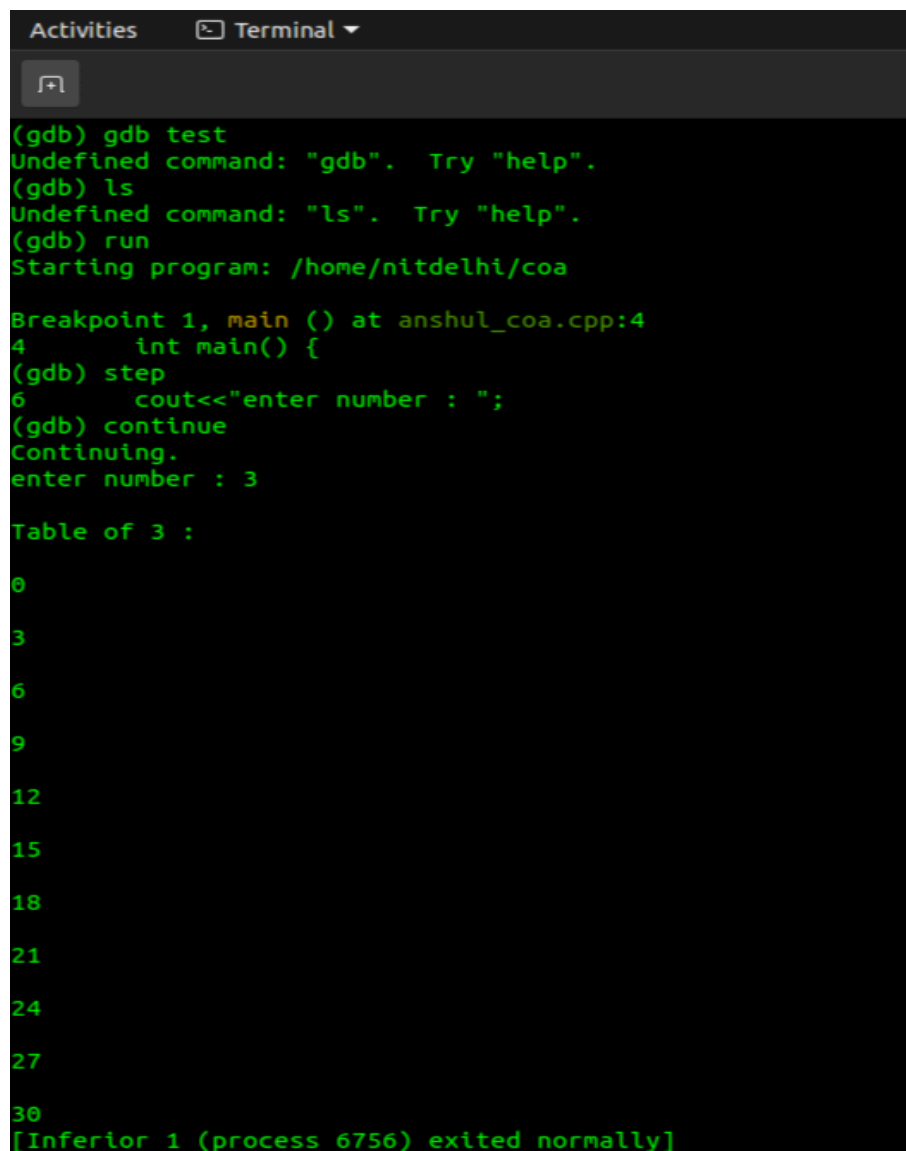
4. Execution Analysis

- Program was compiled successfully with debugging symbols
- Breakpoint was placed at `main()`

- Execution paused before user input
- Step command was used to monitor:
 - Variable declaration
 - User input
 - Loop execution
 - Output generation

Code output :-

Command used - “step”, “run”, “breakpoint main”, “continue”



```
Activities Terminal
(gdb) gdb test
Undefined command: "gdb". Try "help".
(gdb) ls
Undefined command: "ls". Try "help".
(gdb) run
Starting program: /home/nitdelhi/coa

Breakpoint 1, main () at anshul_coa.cpp:4
4      int main() {
(gdb) step
6      cout<<"enter number : ";
(gdb) continue
Continuing.
enter number : 3

Table of 3 :

0
3
6
9
12
15
18
21
24
27
30
[Inferior 1 (process 6756) exited normally]
```

```
(gdb) run
Starting program: /home/nitdelhi/coa

Breakpoint 1, main () at anshul_coa.cpp:4
4      int main() {
(gdb) step
6      cout<<"enter number : ";
(gdb) step
7      cin>>n;
(gdb) print n
$1 = -6416
(gdb) step
enter number : 3
8      cout<<"\n";
(gdb) step

9      cout<<"Table of "<<n<<" : \n"<<endl;
(gdb) print n
$2 = 3
(gdb) contine
Undefined command: "contine". Try "help".
(gdb) step
Table of 3 :

10     for(int i=0;i<10;i++){
(gdb) continue
Continuing.
0

3

6

9

12

15

18

21

24

27

30
[Inferior 1 (process 6892) exited normally]
(gdb) quit
nitdelhi@nitdelhi-HP-Elite-Tower-600-G9-Desktop-PC:~$
```